

# polilabs

## An Architecture Walkthrough

The corpus, the knowledge graph, the agent tools — and who it is for

Internal engineering documentation

May 21, 2026 · companion file: [polilabs-graph.html](#)

**How to read this.** Sections 1–5 explain the machine: what is in the corpus, how a pile of bills becomes a queryable graph, and how an AI agent asks it questions. Section 6 steps back to the human side — who actually uses legislative research and how. Section 7 is an honest status report. The companion file [polilabs-graph.html](#) is the live, clickable version of Figure 2; open it side by side.

---

## Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>What polilabs is</b>                            | <b>2</b> |
| 1.1      | The problem it exists to solve . . . . .           | 2        |
| <b>2</b> | <b>The corpus</b>                                  | <b>3</b> |
| 2.1      | What is in it . . . . .                            | 3        |
| 2.2      | Anatomy of a bill on disk . . . . .                | 3        |
| 2.3      | How bills got in: the ingest pipeline . . . . .    | 4        |
| <b>3</b> | <b>From documents to a graph</b>                   | <b>4</b> |
| 3.1      | Two stores, and why . . . . .                      | 5        |
| 3.2      | The node types . . . . .                           | 5        |
| 3.3      | The edge types — and how each is derived . . . . . | 5        |
| 3.4      | The graph by example . . . . .                     | 6        |
| <b>4</b> | <b>The MCP and the agent tools</b>                 | <b>7</b> |
| 4.1      | What “MCP” means here . . . . .                    | 7        |
| 4.2      | The twelve tools . . . . .                         | 7        |
| 4.3      | How a tool call is implemented . . . . .           | 7        |
| <b>5</b> | <b>End to end: a question becomes an answer</b>    | <b>8</b> |

|     |                                                                 |    |
|-----|-----------------------------------------------------------------|----|
| 6   | Who uses this, and how                                          | 9  |
| 6.1 | A sixty-second primer on bills . . . . .                        | 9  |
| 6.2 | Who does this work . . . . .                                    | 10 |
| 6.3 | How a policy researcher does a bill literature review . . . . . | 10 |
| 6.4 | Where polilabs fits . . . . .                                   | 11 |
| 7   | Honest status — what is built, what is not                      | 11 |

---

# 1 What polilabs is

polilabs is an **agent-native, queryable knowledge graph of US federal legislation**. The version 1 corpus is **191 AI-governance bills** from the 118th and 119th Congresses (2023–2026).

The single most important framing point is this: *the product is not the data*. Anyone can download bill text from Congress.gov. The product is the **interface** — a set of precise, machine-friendly query primitives that let an AI agent investigate the corpus without making things up. polilabs is the backend; the agent doing the research is *yours* (Claude, a Cursor session, a custom app), not polilabs’ own.

## 1.1 The problem it exists to solve

Commercial legal-AI tools hallucinate. Not occasionally — measurably. A 2024 Stanford study (Magesh et al.) found purpose-built legal AI products produced hallucinated answers in **17–34%** of queries; a companion paper (Dahl et al., *Large Legal Fictions*) found near-chance accuracy on questions about whether two legal sources agree.

The polilabs wager is that this happens not because language models are hopeless, but because the *substrate* they reason over is hostile to machines. Legislative text is written for trained humans with a keyword-search box. Hand an agent a search box and ask “does bill X conflict with bill Y” and it will confabulate — because nothing in the substrate lets it *traverse* from X to Y along a typed, verifiable path.

polilabs rebuilds the substrate. It does three things a raw pile of XML cannot:

1. **Reconciliation.** Bill metadata (Congress.gov), full text (GovInfo), and references to existing law (US Code) are stitched into one graph. Each `sources/*` client is deliberately thin — the value is the stitching, not the fetching.
2. **Agent-native primitives.** A question like “which bills define *foundation model*” is *one* tool call, not fifty. The tool set is designed against documented LLM failure modes (the “N+1” search-then-loop pattern, context degradation, pagination truncation).
3. **Anti-hallucination by construction.** Every fact an agent can retrieve carries verbatim provenance. Definitions are scoped to the bill that defines them — there is no global “AI” node — so an agent reporting a definition physically cannot blend two bills together.

## 2 The corpus

### 2.1 What is in it

The v1 corpus is a curated, version-controlled set of 191 bills. “Curated” matters: this is not every bill that mentions AI, it is the set that survived an explicit inclusion test (Section 2.3).

| Dimension                | Count | Dimension              | Count  |
|--------------------------|-------|------------------------|--------|
| Bills (total)            | 191   | House bills (hr)       | 109    |
| 118th Congress (2023–24) | 100   | Senate bills (s)       | 81     |
| 119th Congress (2025–26) | 91    | Concurrent resolution  | 1      |
| Tier A (AI is headline)  | 184   | Sections (graph nodes) | 29,616 |
| Tier B (AI is a part)    | 7     | Defined terms          | 1,241  |
| Cosponsor links          | 688   | Amendment operations   | 190    |

Table 1: The v1 corpus at a glance. Date span of introduction: 2023-01 to 2026-05.

**Subject.** Every bill is AI-governance legislation. The largest policy areas are *Science, Technology, Communications* (62 bills), *Government Operations* (31), *Armed Forces and National Security* (22), and *Commerce* (19) — AI policy is not one committee’s problem, it is spread across the whole legislative map.

**Streams.** The corpus is partitioned by *type of legal instrument*. Only the legislation stream (bills and resolutions) is in v1 scope. The regulatory stream (FTC, NIST, Commerce actions) and the executive stream (executive orders) are deliberately out of scope and kept in separate buckets — they are *never* silently merged in. This is itself an anti-hallucination decision: an agent must be able to say “executive orders are outside this corpus” rather than guess.

**Tiers.** Each bill carries a *centrality tier*. Tier A means AI is the headline subject (the title or summary makes AI policy the core purpose). Tier B means AI is a substantial part but not the headline — e.g. a defense authorization bill with an AI title. Tier C (AI mentioned only in passing) is defined only to mark the *exclusion* boundary: Tier C bills never enter the corpus.

### 2.2 Anatomy of a bill on disk

The corpus is plain files, committed to the repository. Each bill is one directory under `data/corpus/legislation/`, named by its canonical id:

`data/corpus/legislation/118-hr-6881/`

**The id scheme.** A bill’s canonical id is {congress}-{type}-{number} — e.g. 118-hr-6881, 118-s-4178. Inside the graph the same bill is addressed as a URN: `bill:us/118/hr/6881`. A section within a bill is `{bill}::{xml_id}`, where `xml_id` is the stable identifier the legislative XML already carries. The API accepts every form and normalizes at the boundary.

**Inside `bill.xml`.** A bill is a nested tree. The body (`<legis-body>`) contains `<section>` elements, each of which contains `<subsection>`, `<paragraph>`, `<subparagraph>`, `<clause>`... each level carrying an `<enum>` (its label: “1.”, “(a)”, “(1)”), a `<header>` (its heading), and `<text>` (the verbatim provision).

| File            | What it holds                                                                                                                                                                                                   |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| metadata.json   | Reconciled bill metadata: title, sponsor, cosponsors, introduced date, policy area, every legislative action, the list of available text versions, the chosen canonical version, the centrality score and tier. |
| bill.xml        | The full legislative text, as legislative XML (USLM, or the older “pre-USLM” dialect). This is the authoritative text.                                                                                          |
| bill.htm        | The same text as human-readable HTML (a convenience copy).                                                                                                                                                      |
| provenance.json | The audit trail: the exact source URLs (GovInfo XML, GovInfo HTML, Congress.gov), the package id of the version used, and the UTC fetch timestamp. Every fact in the corpus traces back through this file.      |

Table 2: The four-file layout of every bill. 191 directories, identical shape.

Cross-references to existing law appear inline as `<external-xref>` elements. This nested-enum tree, with its stable ids, is exactly what becomes the [Section](#) nodes of the graph.

## 2.3 How bills got in: the ingest pipeline

The corpus is not hand-picked. It is the output of a four-stage pipeline (`ingest/`), and the pipeline’s selectivity is the point.

1. **Discover.** Full-text search against the GovInfo BILLS collection over the date range, with a deliberately *wide* term net (“artificial intelligence”, “machine learning”, “generative AI”, “automated decision systems”, “facial recognition”...). Cast wide now; filter strictly later.
2. **Score.** Each unique bill is run through a hard *anchor gate*: it must contain “artificial intelligence”, “machine learning”, or the standalone token “AI” in a meaningful field. No anchor  $\Rightarrow$  out, no matter how AI-adjacent it looks. Survivors get a *centrality score*: a weighted sum over where the anchor terms appear (a hit in the title counts  $4\times$  a hit in the summary).
3. **Reconcile.** For each survivor, rich metadata is pulled from Congress.gov — sponsor, cosponsors, actions, policy area, official summary — and cross-checked against the GovInfo text. “Cross-check, don’t wrap.”
4. **Promote.** Bills scoring above threshold (or marked include by a human reviewer) are materialized into the four-file directory layout, and the corpus index is written.

The funnel is real: **279 candidate bills were discovered; 191 were promoted.** The other 88 fell below the centrality threshold. The pipeline is re-runnable and idempotent, so the corpus can be refreshed or expanded without disturbing what is already in.

## 3 From documents to a graph

A directory of XML files is authoritative but not queryable. Turning it into something an agent can traverse is the engineering core of polilabs. It happens in *two* stores, built side by side from the same source.

### 3.1 Two stores, and why

|                | SQLite (polilabs.db)                                                          | Kùzu graph (polilabs.kuzu)                                                     |
|----------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>Shape</b>   | Relational tables + full-text search index                                    | Property graph: typed nodes, typed edges                                       |
| <b>Good at</b> | “Find bills mentioning X” — fast keyword discovery; bill and section metadata | “What connects to what” — definitions, citations, amendments, cross-bill links |
| <b>Holds</b>   | bills, sections, cosponsors, actions; two FTS5 search indexes                 | Bill, Section, DefinedTerm, StatuteSection, AmendmentOperation...              |

Table 3: The two stores. Both are *embedded* (in-process, no server, like a file) and both are regenerated destructively from `data/corpus/`. They are not redundant: discovery wants a search index, structural reasoning wants a graph.

Both are derived; neither is a source of truth. Delete them and they rebuild from the corpus in about a minute. Figure 1 shows the whole path.

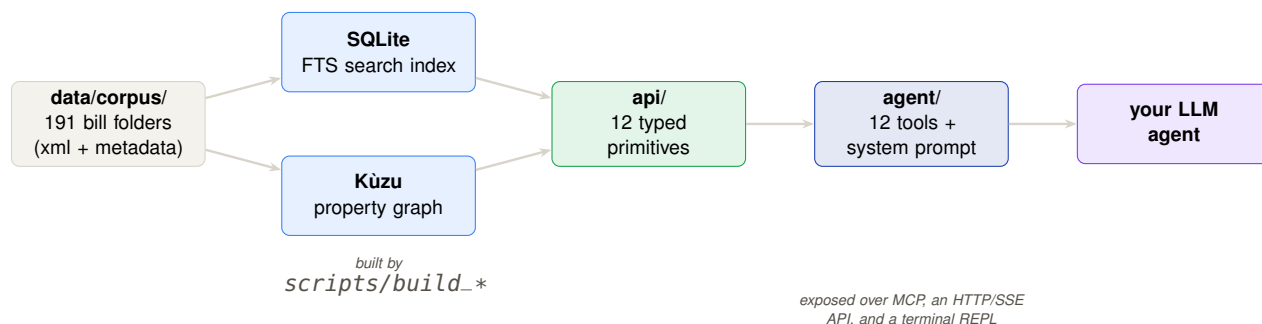


Figure 1: The pipeline. The corpus is the source of truth; everything to its right is derived and regenerable.

### 3.2 The node types

The graph has ten node types that are populated in v1. Think of them in three groups: the *bibliographic* backbone (the bill as a document), the *substantive* layer (what the bill says and does), and the *provenance* spine (who derived what).

### 3.3 The edge types — and how each is derived

Edges are where the value is. Each is *typed* (it has a name and a fixed source/target), and each is *derived mechanically* from the source XML — no language model invents an edge.

A few of the derivations are worth spelling out, because they are where “mechanical, not invented” is earned:

- **DEFINES.** The extractor finds containers whose heading matches “Definitions”, and treats each

| Node               | Group  | What it is                                         | Count  |
|--------------------|--------|----------------------------------------------------|--------|
| Jurisdiction       | biblio | The sovereign (here: the United States)            | 1      |
| Bill               | biblio | The bibliographic record of one bill               | 191    |
| BillVersion        | biblio | A dated text snapshot at one procedural stage      | 191    |
| Section            | biblio | A node in the bill’s body tree (section... clause) | 29,616 |
| Sponsor            | biblio | A legislator who (co)sponsors a bill               | 411    |
| DefinedTerm        | subst. | A term a section formally defines                  | 1,241  |
| StatuteSection     | subst. | A section of existing US Code a bill touches       | 172    |
| AmendmentOperation | subst. | One reified edit to existing law                   | 190    |
| Extractor          | prov.  | A parser that derived edges                        | 4      |
| ProvenanceRecord   | prov.  | One record of an extractor run                     | 4      |

Table 4: The populated node types. (Three further types — Committee, Statute, UnresolvedTermUse — are declared in the schema but not yet populated; see Section 7.)

| Edge            | From → To                           | How it is derived                                 | Count  |
|-----------------|-------------------------------------|---------------------------------------------------|--------|
| OF_JURISDICTION | Bill → Jurisdiction                 | Every bill, mechanically                          | 191    |
| HAS_VERSION     | Bill → BillVersion                  | From the chosen canonical version                 | 191    |
| HAS_SECTION     | BillVersion → Section               | Each top-level section of the body                | 647    |
| PARENT_OF       | Section → Section                   | The nesting of the body tree                      | 28,969 |
| SPONSORED_BY    | Bill → Sponsor                      | From the bill’s sponsor field                     | 191    |
| COSPONSORED_BY  | Bill → Sponsor                      | From the cosponsor list                           | 688    |
| DEFINES         | Section → DefinedTerm               | Sections under a “Definitions” heading            | 1,244  |
| BY_REFERENCE    | DefinedTerm → StatuteSection        | A definition that says “has the meaning given...” | 114    |
| CITES_EXTERNAL  | Section → StatuteSection            | From <external-xref> citations in the XML         | 646    |
| AMENDS          | Section → AmendmentOperation        | One per “quoted-block” edit in the text           | 190    |
| TARGETS         | AmendmentOperation → StatuteSection | The US Code section the edit modifies             | 182    |
| PRODUCED        | Extractor → ProvenanceRecord        | The trust spine                                   | 4      |

Table 5: The twelve populated edge types. The two big ones (PARENT\_OF, HAS\_SECTION) are the body tree; the interesting ones (DEFINES, BY\_REFERENCE, AMENDS, TARGETS, CITES\_EXTERNAL) are the substantive layer that makes cross-bill reasoning possible.

child as one defined term. The term’s surface form and verbatim definition text are copied, not paraphrased.

- **BY\_REFERENCE.** When a definition’s text matches a pattern like “has the meaning given such term in...” and contains a US Code citation, the definition is linked straight to that statute section. The bill is borrowing a definition; the graph records the borrowing.
- **AMENDS / TARGETS.** Each <quoted-block> (the XML marker for inserted or replacement text) becomes one AmendmentOperation node. Its operation type (*strike*, *insert*, *replace*...) is classified from the surrounding prose; its target US Code section is the nearest enclosing citation. The amendment is a *node*, not a label, so “what would the statute read like if this passes” is a computable walk, not a narrative guess.

### 3.4 The graph by example

Figure 2 is the whole idea in one picture. It shows two real bills from the corpus and the single statute node that binds them.

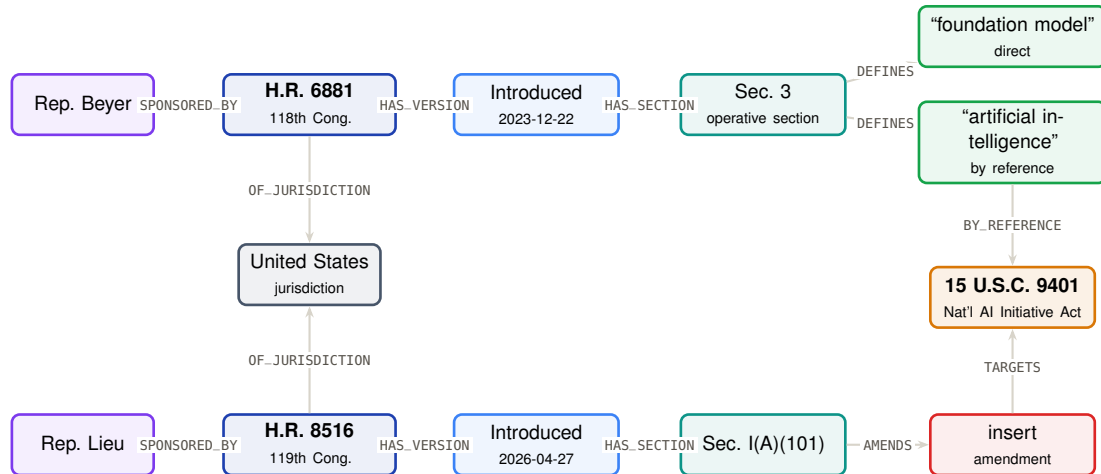


Figure 2: Two real bills, one shared statute. **H.R. 6881** borrows its definition of “artificial intelligence” from **15 U.S.C. 9401** (BY\_REFERENCE); **H.R. 8516** rewrites that same section (AMENDS → TARGETS). Two bills that share no sponsor, no Congress and no keyword are bound together through one statute node — and the graph finds that link in a single hop. The companion file `polilabs-graph.html` is this figure, live and clickable.

This is the payoff of the whole architecture. “Which bills touch the definition of AI that Congress enacted in 2020?” is, against a pile of XML, a research project. Against the graph it is one query: find 15 U.S.C. 9401, follow every BY\_REFERENCE and TARGETS edge backwards.

## 4 The MCP and the agent tools

### 4.1 What “MCP” means here

MCP — the Model Context Protocol — is a standard way to expose a set of *tools* to an LLM. A tool is a named function with a typed schema: the model sees “here is a function `search_corpus(query, tier, limit)`”, decides to call it, and gets a structured result back.

polilabs ships an MCP server (`mcp_server.py`). Point Claude Desktop, Cursor, or any MCP client at it and that client’s model gains 12 new tools for querying federal AI legislation. The MCP server needs no API key of its own — it only reads the two local stores; the LLM is supplied by the client.

Critically, **the same 12 tools back four different front doors**: the MCP server, an HTTP/SSE web API, a terminal REPL, and the evaluation harness. They all import one tool catalogue and one system prompt from `agent/tools.py`. Write a tool once, get it everywhere.

### 4.2 The twelve tools

### 4.3 How a tool call is implemented

Every tool is the thin top of a three-layer stack:

| Tool                                                               | What it does                                                               | Store          |
|--------------------------------------------------------------------|----------------------------------------------------------------------------|----------------|
| <i>Discovery &amp; scope</i>                                       |                                                                            |                |
| search_corpus                                                      | Free-text search; ranked bill hits + a hint that routes to aggregate tools | SQLite         |
| corpus_coverage                                                    | What is in and out of scope — the “say I don’t know” tool                  | SQLite         |
| <i>Single-bill drill-in</i>                                        |                                                                            |                |
| get_bill                                                           | Metadata + a table of contents (no body text)                              | SQLite         |
| get_section                                                        | Verbatim section text + its canonical citation                             | SQLite + graph |
| get_defined_terms                                                  | Every term one bill formally defines                                       | graph          |
| get_amendments                                                     | Every amendment one bill makes to existing law                             | graph          |
| <i>Cross-reference &amp; targeting</i>                             |                                                                            |                |
| get_citation_graph                                                 | The typed citation graph around a section                                  | graph          |
| get_amendments_targeting                                           | Every amendment in the corpus hitting one US Code section                  | graph          |
| resolve_citation                                                   | “Sec. 3(a) of H.R. 1736” → a canonical section id                          | SQLite         |
| <i>Aggregate — “list every bill that...” (one call, not fifty)</i> |                                                                            |                |
| find_bills_defining                                                | Every bill defining a given term                                           | graph          |
| find_bills_amending                                                | Per-bill rollup of bills amending a US Code section                        | graph          |
| find_definitions_of                                                | Every bill’s verbatim definition of a term, side by side                   | graph          |

Table 6: The agent tool surface. The four *aggregate* tools were added after evaluation revealed that agents systematically truncate “search-then-loop” patterns at roughly 20 sequential calls — so the expensive multi-step pattern was collapsed into a single primitive.

1. **Tool wrapper** (`agent/tools.py`) — accepts plain JSON arguments, validates and clamps them, calls the primitive below, and serializes the result to a JSON string. Errors become structured `{"error": ...}` objects; the agent never sees a stack trace.
2. **Typed primitive** (`api/_impl.py`) — the real query logic. It runs SQL against SQLite, or Cypher against the Kùzu graph, or both, and returns a typed Python object.
3. **Storage** — the SQLite FTS index and the Kùzu graph.

The contract for that middle layer is written down in `api/SPEC.md`, and its invariants are themselves anti-hallucination rules. Three of the nine: every chunk of verbatim text is returned *with* its canonical citation, so the agent never reconstructs a citation; an empty result is always distinguishable from an out-of-scope result; and a primitive *never* raises on “not found” — it returns a typed answer that says so.

## 5 End to end: a question becomes an answer

Putting it together — here is what happens when a researcher asks, through the web front end, “How does each bill define ‘foundation model?’”

1. The question hits the `POST /chat` endpoint. The server starts an agent loop with **Claude Opus 4.7** and the 12 tools.
2. The system prompt tells the agent: for any “which bills do X” question, do *not* search then loop — go straight to an aggregate tool. So the agent calls `find_definitions_of("foundation model")`.



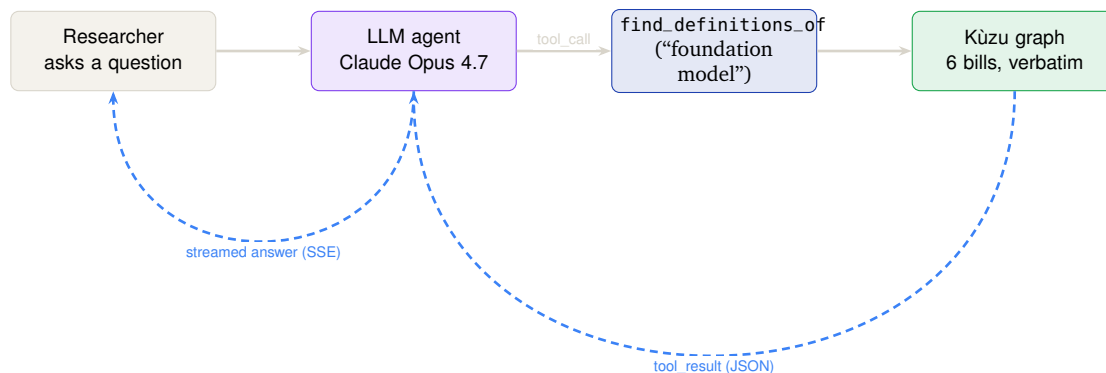


Figure 3: The chat path. The agent loop runs until the model stops calling tools; the server streams every step.

3. That one call runs a single graph query and returns every bill’s *verbatim* definition of the term — six of them — side by side.
4. The agent writes the answer, quoting the definitions and their canonical citations exactly as the tool returned them. It cannot drift, because it never had the raw text in any other form.
5. The whole run streams to the browser as Server-Sent Events: `text` (answer prose), `tool_call` (a tool was invoked), `tool_result` (structured data, so the UI can render panels), `done`, `error`. The front end renders the answer and the sources as they arrive.

The web front end also has a second, cheaper path: a set of read-only `GET /api/*` REST endpoints that wrap nine of the tools directly. Clicking between bills in the UI uses those — no LLM, no token cost, instant.

## 6 Who uses this, and how

The machine above only matters if it shortens real work. This section is for the reader with no policy background.

### 6.1 A sixty-second primer on bills

A **bill** is a proposed law. A member of the House introduces an “H.R.” bill; a Senator introduces an “S.” bill; each gets a number (H.R. 6881). It is referred to a committee. *Most bills die there* — they are never voted on. A few are amended (“marked up”), voted out of committee, passed by the full chamber, sent to the other chamber, and — rarely — signed into law.

Two facts shape the research problem:

- **Congresses are two years long and bills do not survive them.** The 118th Congress was 2023–24; the 119th is 2025–26. A bill not passed by the end of its Congress is dead and must

be reintroduced from scratch — often with a new number and slightly changed text. So the “landscape” on any topic is dozens of overlapping, competing, reintroduced drafts.

- **Bills mostly edit existing law.** A bill rarely writes a statute from a blank page. It *amends* the US Code — “strike subsection (b) and insert the following”. To understand a bill you must understand what it changes. And bills **define their own terms**: “artificial intelligence” means whatever *this* bill’s definitions section says, and the next bill may say something different.

That last point is why polilabs scopes definitions to bills and never builds a global “AI” node. The disagreement between definitions is the research finding.

## 6.2 Who does this work

| User                           | What they need from legislative research                                                                                                                                                               |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Congressional staff            | Legislative assistants and committee staff drafting a bill: what else has been introduced on this topic, who sponsored it, what it does, what it would amend — so they do not duplicate or contradict. |
| CRS analysts                   | The Congressional Research Service is Congress’s in-house, nonpartisan research arm. It writes side-by-side comparisons of legislative approaches for members of both parties.                         |
| Government affairs / lobbyists | Track every bill that could affect a client or industry; need early warning and precise, defensible analysis.                                                                                          |
| Think tanks & academics        | Groups like CSET, Brookings, or Stanford HAI publish “the state of federal AI legislation” reports — exactly a literature review over bills.                                                           |
| Corporate policy & compliance  | AI companies especially: what regulation is coming, how would each bill’s definitions catch (or exempt) our products.                                                                                  |
| Journalists & advocacy orgs    | Cover or campaign on specific bills; need to get the facts and citations exactly right.                                                                                                                |

Table 7: The audience. All of them do some version of the same task below.

## 6.3 How a policy researcher does a bill literature review

Suppose a researcher is asked: “*Give us the landscape of how federal AI bills define and regulate foundation models.*” Today, without polilabs, the work runs roughly like this:

1. **Scope.** Pin down the question — which Congresses, which sense of the term, what counts as “in”.
2. **Discover.** Search Congress.gov (free, official) or a paid platform — Bloomberg Government, POLITICO Pro, Quorum, FiscalNote — by keyword, policy area, and subject. This returns a long, noisy list.
3. **Triage.** Read each title and its official summary; skim; cull the list to what is genuinely relevant. Slow, and judgment-heavy.

4. **Deep read.** Pull the full text of each surviving bill. Find the definitions section. Find the operative sections. Work out what existing law it amends.
5. **Compare.** Build a comparison — almost always a spreadsheet: one row per bill, columns for the definition, the threshold, the scope, the enforcing agency, the penalties. This matrix is the deliverable, and building it is entirely manual.
6. **Write up.** Turn the matrix into a memo, with every bill and section cited exactly — because a wrong citation discredits the whole document.

The pain is concentrated in steps 3–6. Keyword search (step 2) silently misses bills that use a different phrase. Because every bill defines terms locally (step 4), the comparison in step 5 is apples-to-oranges by construction — you cannot compare “foundation model” across bills until you have read each bill’s own definition. And step 6 is where general-purpose AI tools fail worst: asked to summarize, they hallucinate citations.

## 6.4 Where polilabs fits

polilabs is not a chat-with-a-PDF wrapper bolted onto this workflow. It replaces the *substrate* the workflow runs on, so that steps 2–5 become tool calls an agent makes in seconds:

- Discovery and triage (steps 2–3) → `search_corpus`, with a curated corpus that already applied the inclusion test.
- The comparison matrix (step 5) → `find_definitions_of("foundation model")` returns every bill’s verbatim definition side by side — the spreadsheet, in one call.
- “What does each bill change about existing law” → `get_amendments` and `find_bills_amending`.
- The write-up (step 6) is safe *by construction*: every definition, every citation the agent reports came verbatim from a tool result, carrying its own provenance. The agent can also say, honestly, “executive orders are outside this corpus” instead of guessing.

The researcher still does the thinking — scoping the question, judging significance, writing the argument. polilabs removes the mechanical substrate-fighting in the middle, and removes the hallucination risk at the end.

## 7 Honest status — what is built, what is not

This document describes a v1 that genuinely works, but it would be misleading to imply it is finished.

**What works.** All 191 bills are ingested, parsed, and indexed into both stores with zero parse errors. All 12 tools are implemented and live across all four transports. The evaluation harness — 13 hand-curated queries scored for both under-coverage and over-confidence — passes **12 of 13**, with zero hallucinated citations; the one failure is model variance on a stylistic check, not a code bug.

**What is stubbed.** The schema design document (`schema_design.md`) specifies a far larger ontology — 23 node types, interpretive-reasoning nodes, bitemporal versioning, soft “conflicts-with” edges,

an enactment-version synthesizer. The *implemented* graph is the mechanical subset: ten populated node types, twelve populated edge types. Three declared node types (Committee, Statute, UnresolvedTermUse) and four edge types are not yet populated. Treat `schema_design.md` as the roadmap and this document as the territory.

**Known limits.** v1 stores one canonical version per bill — the schema supports a full version history, but it is not wired. Amendment “before” text is marked unverified, because the US Code itself is not yet ingested for comparison. The two USLM-format bills produce no citation edges (a parser gap). And the corpus is, deliberately, only AI-governance legislation — not all of federal law.

None of this undercuts the v1 thesis. It bounds it: polilabs v1 is a working, honest, anti-hallucination research substrate for one well-defined corpus, with a clear path to more.

---

Companion: open `polilabs-graph.html` for the interactive graph.  
Source of every number here: the v1 corpus and the indexes built from it.